Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · LAB 3

Wildcards, Links, GRUB and Shared Libraries

Lab 3 guide: select files with patterns, then see what a rename does to a hard link and to a symbolic link.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Select files with a wildcard and explain shell expansion versus a literal name.
2. Create and inspect hard links and symbolic links, using inode and target-path evidence.
3. Predict and diagnose what renaming or replacing a link target does.

THIS LAB

Lab 3

First the concepts, with examples. Then the lab on the server. GRUB and shared libraries are optional extensions, so they come at the end.

DISCUSSION

You rename a report. What breaks?

A shortcut on the desktop, a second name for the same file, a note that says where the file is. Which of them still work?

CONCEPTS FIRST

Wildcards and links

Two ideas: the shell turns patterns into names, and one file can have more than one name.

A wildcard is a pattern, and the shell expands it

- A wildcard is a character that stands for other characters, such as * for any characters.
- The shell, not the command, expands the pattern into a list of matching names, then runs the command with that list.
- If nothing matches, Bash keeps the pattern as typed. The command then looks for a file with that exact name.
- Wildcards select file names. They are not the same as regular expressions.

BEFORE THE LAB

Why learn it here?

One pattern replaces a long list of names. The same pattern, used carelessly with rm, removes more than you meant.

The wildcard patterns

Pattern	Meaning	Example	Matches
`*`	Zero or more characters	`*.txt`	every .txt file
`?`	Exactly one character	`file?.txt`	file1.txt, fileA.txt
`[abc]`	One character from the set	`file[123].txt`	file1.txt, file2.txt, file3.txt
`[a-z]`	One character in the range	`[A-Z]*.txt`	names that start with a capital
`[!abc]`	One character not in the set	`file[!0-9].txt`	fileA.txt, not file1.txt
`{a,b}`	Brace expansion: generates words	`file.{txt,log}`	the words file.txt and file.log

Square brackets match one character that exists. Braces only write words, before any file is looked at.

Patterns at work

student@server: ~/oslab-work/lab3

```
$ ls *.txt
$ ls report0[1-3].txt
$ ls log_[!j]*.log
$ touch memo_{mon,tue,wed}.txt
$ cp *.csv archive/
$ ls *.tmp
$ rm *.tmp
```

The first three list files. touch with braces creates three files in one command.

cp copies every .csv file into archive/.

CAUTION

Preview before you delete

Run ls *.tmp first. If the list is what you expect, then run rm *.tmp.

See what the shell really passes

student@server: ~/oslab-work/lab3

```
$ mkdir -p practice
$ printf 'one\n' > practice/report1.txt
$ printf 'two\n' > practice/report2.txt
$ printf 'log\n' > practice/report.log

$ printf '<%s>\n' practice/*.txt
<practice/report1.txt>
<practice/report2.txt>
$ printf '<%s>\n' 'practice/*.txt'
<practice/*.txt>
```

printf prints each word it receives between angle brackets, one per line.

Without quotes, the shell sends two words: the two .txt files. With single quotes around the whole pattern, it sends one word with a literal *.

Use this trick to check a pattern before you give it to cp, mv or rm.

Brackets and braces are not the same

Square brackets look at files

```
student@server: ~/oslab-work/lab3

$ ls practice/report[12].txt
practice/report1.txt  practice/report2.txt
$ ls practice/report[13].txt
ls: cannot access 'practice/report3.txt': No such file
```

Only names that exist can match. Bash keeps a pattern with no match, so the error names the pattern.

Braces only write words

```
student@server: ~/oslab-work/lab3

$ echo practice/report{1,3}.txt
practice/report1.txt practice/report3.txt
```

The file report3.txt does not exist, and echo does not care. Brace expansion happens first, and no file is looked at.

A name is not the file

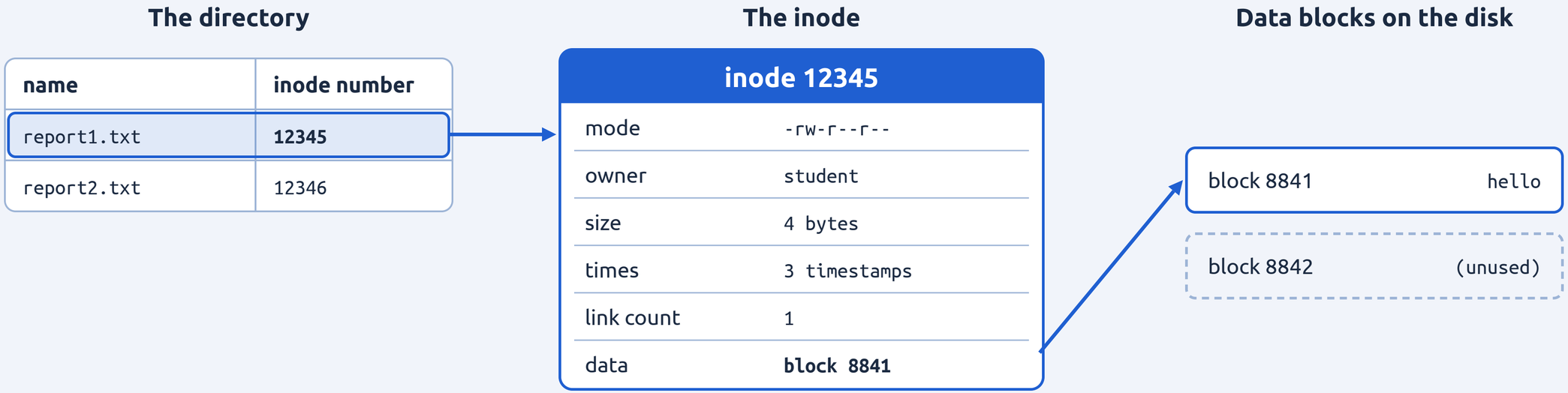
- The data of a file lives on the disk. An inode is the record that describes it: size, owner, permissions and where the data is.
- Every inode has a number. A file name is only an entry in a directory that points to an inode number.
- Two entries can point to the same inode. They are two names for one file: hard links.
- The data is freed only when the last name is gone.

inode number, permissions, link count, ...

```
$ ls -li
12345 -rw-r--r-- 2 student student 4 report1.txt
12345 -rw-r--r-- 2 student student 4 hard-demo.txt
99001 lrwxrwxrwx 1 student student 11 soft-demo.txt -> report1.txt
```

The 2 in the third column is the link count: two names point to inode 12345. The symbolic link has its own inode, 99001.

What an inode looks like



The directory maps a name to an inode number. The inode describes the file and says where its data is.

The name is not stored in the inode.

Read it from left to right: the name leads to an inode number, the inode leads to the data. ls -li shows that number.

Hard link and symbolic link

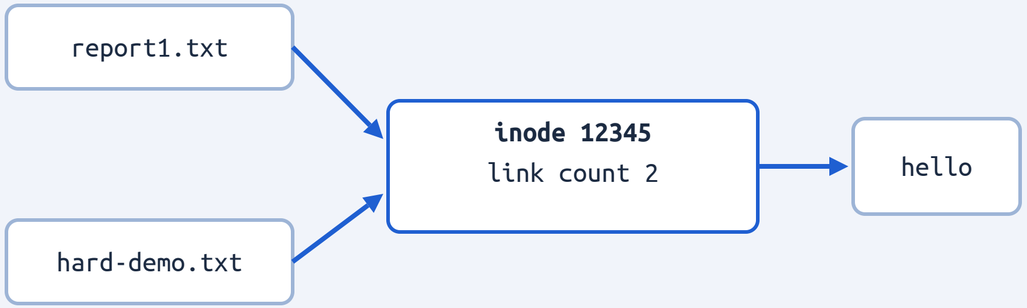
	Hard link	Symbolic link
Command	`ln target link`	`ln -s target link`
What it is	Another name for the same inode	A small file that stores a path
Inode	The same as the original	Its own inode
Original is removed	Data survives under the other name	The link breaks: it is dangling
Across filesystems	No	Yes
Folders	Not normally	Yes
Seen in ls -l	No arrow	An arrow to the stored target

Inspect with ls -li (inode numbers), stat (link count) and readlink (the stored path).

The two kinds of link, drawn

Hard link

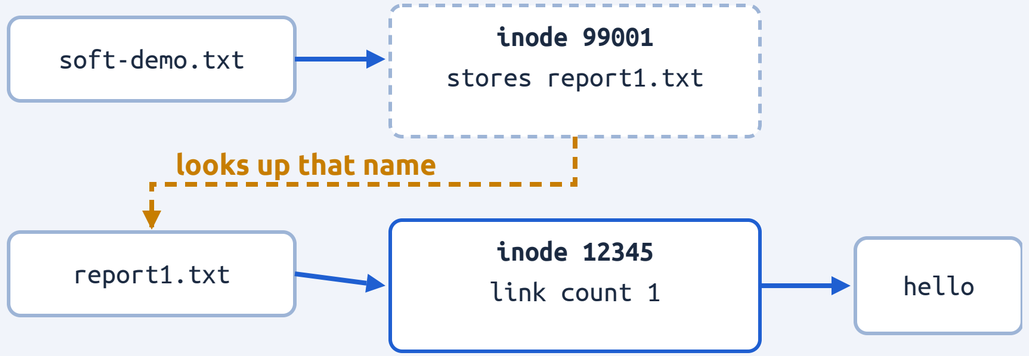
`ln report1.txt hard-demo.txt`



Two names for one inode.
Remove one name and the data stays.

Symbolic link

`ln -s report1.txt soft-demo.txt`



Its own inode, storing a path.
Rename the target and the path breaks.

A hard link is a second name for the inode. A symbolic link is a separate small file that holds a name to look up.

A relative target is read from the link's folder

student@server: ~/oslab-work/lab3

```
$ cd ~/oslab-work/lab3
$ ls -l links
lrwxrwxrwx 1 student student 11 soft.txt -> source.txt

$ cat links/soft.txt
version 1
$ cd links && cat soft.txt
version 1
```

The stored target source.txt has no / at the front, so it is relative.

It is resolved from the folder that holds the link (links/), not from the folder where you are standing.

That is why the same link works from two places. Write the target as if you were standing next to the link.

The key experiment

student@server: ~/oslab-work/lab3

```
$ echo 'config data' > original.txt
$ ln original.txt hardlink.txt
$ ln -s original.txt symlink.txt
$ ls -li
$ rm original.txt
$ cat hardlink.txt
config data
$ cat symlink.txt
cat: symlink.txt: No such file or directory
```

echo with > writes the text into a new file. rm removes only the name original.txt.

The data still has one name, hardlink.txt, so it survives.

symlink.txt still says original.txt, and no such name exists. The link is dangling.

In practice: ls -la /usr/bin/python* shows programs that are managed with symbolic links.

LAB 3

Wildcards and links

A file is renamed. The references that depend on it must still work, or you must say why they do not.

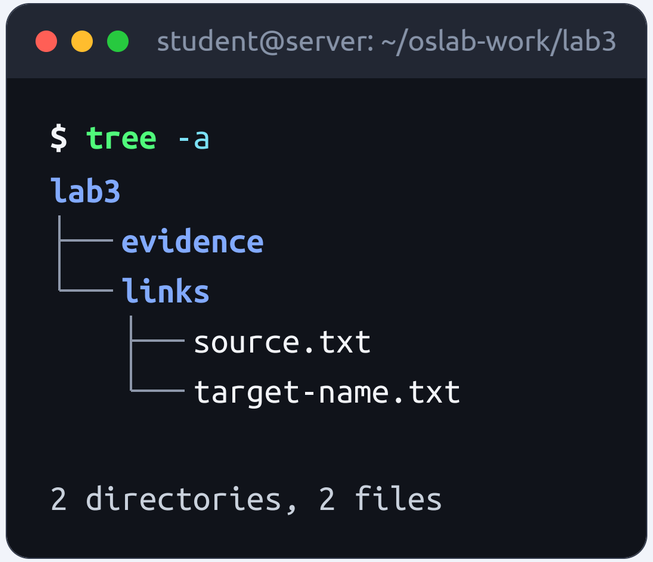
The scenario

SCENARIO

Alex at TechCorp

Alex maintains file references. A file is being renamed, and the references that depend on it must still work.

Wildcards and links are the core. Boot recovery and custom libraries are separate extensions.



Inside the 120 minutes

Minutes	What happens
0 to 10	Introduction, objectives and setup
10 to 25	Guided example: commands and observations
25 to 35	Written prediction
35 to 70	Individual investigation, Task 2

Minutes	What happens
70 to 85	Normal and edge tests, feedback
85 to 100	Individual changed-case checkpoint
100 to 110	Correction and explanation
110 to 120	Cleanup and submission

How you work

Your own account

Log in with your own account. Use only your own files and processes. Privileged and boot work is never done on the shared server.

AI tools

Optional in the investigation and the tests. Not allowed in the prediction and the individual checkpoint.

Evidence

Keep what you observe in the evidence folder. Short, attributable evidence earns more than long output.

Initial prediction accuracy is lightly weighted. A reasoned attempt and an evidence-based correction earn credit.

Setup: start the lab

student@server: ~

```
$ whoami
$ command -v oslab
$ oslab doctor
$ export OSLAB_WORKSPACE="${OSLAB_WORKSPACE:-$HOME/oslab-work}"
$ oslab start lab3
$ cd "$OSLAB_WORKSPACE/lab3"
$ pwd
$ mkdir -p evidence
$ find . -maxdepth 3 -type f
```

- whoami shows your account. command -v oslab and oslab doctor check the helper.
- If the helper is missing, follow the local setup guide in the course repository.
- oslab start lab3 makes the lab folder. It keeps your work if you start again, and it does not change your current folder, so cd into it yourself.
- If you are resuming, look at the existing files before you run a command that writes to them.

Compare with the starting tree

student@server: ~/oslab-work/lab3

```
$ tree -a -L 3
lab3
├── evidence
└── links
    ├── source.txt
    └── target-name.txt

2 directories, 2 files
```

The helper also writes a hidden file, .oslab-managed.json. It marks the folder as managed. Leave it as it is.

evidence/ was made by the mkdir -p in step 3.

The workspace is for experiments. Your personal course repository holds the final submission.

GUIDED

Wildcards, then your first links

Task 1, with the instructor: ten to twenty-five minutes.

Step 1: three clearly different names

student@server: ~/oslab-work/lab3

```
$ mkdir -p practice
$ printf 'one\n' > practice/report1.txt
$ printf 'two\n' > practice/report2.txt
$ printf 'log\n' > practice/report.log
$ ls practice
report1.txt  report2.txt  report.log
```

Work in a separate practice folder, so the investigation files stay untouched.

printf with > writes one line into a new file. The names differ on purpose: two .txt files and one .log file.

Step 2: how the shell expands a pattern



The first two commands name the two text files. The quotes around the whole pattern in the third leave a literal *.

Brace expansion such as {1,2} generates words. It is not a test against file names.

TRY

Observe

Which pattern did not match report.log? Why does report?.txt need exactly one character after report?

Step 3: your first two links

student@server: ~/oslab-work/lab3

```
$ ln practice/report1.txt practice/hard-demo.txt
$ ln -s report1.txt practice/soft-demo.txt
$ ls -li practice/report1.txt practice/hard-demo.txt practice/soft-demo.txt
$ readlink practice/soft-demo.txt
```

A hard link names the same inode, on one filesystem. A symbolic link stores a path. ls -l shows the stored target.

readlink prints the path stored in a symbolic link.

TRY

Observe

Which inode numbers match? Why is report1.txt, not practice/report1.txt, the right target here?

PREDICT

Guess first, then try

Before you rename the investigation source. Ten minutes.

The prediction

PREDICTION

Write this down first

After source.txt becomes renamed.txt, will a hard link still read the old bytes? Will a symbolic link storing source.txt still work? Explain each mechanism.

HOW

The rules

Alone, and without AI. On paper or in the course worksheet, before you run mv. You may compare with a neighbour for five minutes, but keep your own answer.

A wrong prediction costs nothing. It is marked for being made and for the evidence-based correction afterwards.

TASK 2

Hard links and symbolic links

Thirty-five minutes, from minute 35 to 70. Work alone.

Step 1: two references to the source

student@server: ~/oslab-work/lab3/links

```
$ cd "$OSLAB_WORKSPACE/lab3/links"
$ cat source.txt
$ ln source.txt hard.txt
$ ln -s source.txt soft.txt
$ ls -li source.txt hard.txt soft.txt | tee ../evidence/links.txt
```

| tee sends the output to the screen and into evidence/links.txt at the same time. Your first record is the before-state.

CAREFUL

If you are resuming

A destination that already exists must be inspected, not overwritten. Do not run ln again and again: ln refuses with File exists.

Step 2: rename the original, then look

student@server: ~/oslab-work/lab3/links

```
$ mv -- source.txt renamed.txt
$ cat hard.txt
$ readlink soft.txt
$ cat soft.txt 2>&1 | tee -a ../evidence/links.txt
```

mv renames the directory entry. -- means no more options follow.

2>&1 sends the error message to the same place as normal output, so tee can record it. -a appends instead of replacing.

THINK

A deliberate failure

The last command fails on purpose. Write down what the error means before you repair anything.

Step 3: repair only the symbolic link

What to do

1. Look at the old link first: readlink soft.txt.
2. Remove only that owned symbolic-link entry.
3. Create it again with ln -s, and choose a relative target yourself.
4. Do not replace hard.txt. It never broke.

YOUR TURN

You write the target

Nobody gives you the command here. Ask: from the folder links/, what is the name of the file now?

Step 4: inode identity after the repair

student@server: ~/oslab-work/lab3/links

```
$ {
$   ls -li renamed.txt hard.txt soft.txt
$   readlink soft.txt
$   cat hard.txt soft.txt
$ } | tee -a ../evidence/links.txt
```

The braces group four commands, so one tee records all of their output.

TRY

Observe

Which inode numbers match now? Which name did you create again, and which one did not change?

Step 5: a changed case, a new source.txt

student@server: ~/oslab-work/lab3/links

```
$ printf 'version 2\n' > source.txt
$ { ls -li source.txt hard.txt; cat source.txt hard.txt; } | tee ../evidence/replacement.txt
```

Create a new source.txt that contains version 2. Before you run the check, predict: will hard.txt read version 1 or version 2?

Then explain why reusing a file name does not reuse the old inode.

Complete when, and hints

DONE

Complete when

The hard link survives the rename. The repaired symbolic link resolves correctly. Your changed-name test tells a file name from an inode.

Hints, one at a time

1. A hard link is another directory entry.
2. Compare ls -li with readlink.
3. Resolve the symbolic target from links/, not from the folder where you are standing.

Check your work

Fifteen minutes, from minute 70 to 85.

Three cases, three records

Case	Evidence	What it tests
Normal	All references read version 1 before the rename	Building the links and the correct target path
Failure and repair	The symbolic read fails after the rename and works after the repair	Path indirection and diagnosis
Changed name	The new source reads version 2, the hard link keeps version 1	File name replacement versus inode identity

Save the before, renamed and repaired observations in evidence/links.txt, and the recreated-name result as evidence/replacement.txt. Go back to the workspace root first.

Links and the helper

Internal links are fine

The helper accepts symbolic links that stay inside the managed folder.

Escaping links are refused

It refuses links that point outside it. Do not link to /etc, other homes or unrelated folders.

oslab clean lab3

Removes only the managed folder. Internal links go as entries, and external links are refused.

File exists means: look at the destination first. A dangling link can exist even when its target does not. Hard links cannot normally cross filesystems.

INDIVIDUAL CHECKPOINT

Show what you can do alone

Fifteen minutes, from minute 85 to 100. No AI, no neighbours.

Why there is a checkpoint

It checks you, not the tools

Everything before it may use AI and help. The checkpoint shows what you understood without them.

A short question on paper

You answer one question on paper or on the course worksheet. Your earlier work does not need to be finished.

Practice question

The question on the next slide is public practice. A graded session may use a fresh private variant.

The question

PRACTICE QUESTION

Resolve the path

A symbolic link inside lab3/links/ stores ../source.txt. Only lab3/links/source.txt exists. Resolve the stored path and explain why the link fails. Give a suitable target for the existing file.

The instructor collects your answer before discussing the public key.

1. The result or diagnosis.
2. The mechanism.
3. One observation that could check it.

What to give

Look back, then hand in

Ten minutes of correction, ten to clean up and submit.

Correct your prediction

1. Keep your original prediction in view. Under it, write confirmed or corrected.
2. Cite the test that shows it.
3. Explain the OS mechanism in three to five sentences.

ANSWER

Three questions

(a) Which file change broke the symbolic reference? (b) Why did the hard link survive? (c) What happens if the new file name holds different bytes?

Submit 1: your repository and the copy

student@server: ~

```
$ SUBMISSION_REPO="$HOME/os-gic-YOUR_ID/os-lab-YOUR_ID"
$ mkdir -p "$SUBMISSION_REPO/lab3/evidence"
$ cd "$OSLAB_WORKSPACE/lab3"
$ cp -- evidence/links.txt evidence/replacement.txt "$SUBMISSION_REPO/lab3/evidence/"
```

Replace YOUR_ID with your own ID. The orange text is the part you change. Use the absolute path of your existing course repository, never a neighbour's.

Example: with the ID 0123, the first line becomes SUBMISSION_REPO="$HOME/os-gic-0123/os-lab-0123", and echo "$SUBMISSION_REPO" prints /home/student/os-gic-0123/os-lab-0123.

Copy the listed artifacts and two selected test records, not every terminal output.

Submit 2: the final tree and Git

student@server: ~/os-gic-YOUR_ID/os-lab-YOUR_ID

```
$ tree
lab3
├── evidence
│   ├── links.txt
│   └── replacement.txt
└── README.md

1 directory, 3 files
```

1. Write README.md from the lab's template, in your own words.
2. The prediction must have been written before you ran anything. Copying it into the README only records it.
3. git status --short, then add only your lab files, then commit and push.
4. No passwords, personal shell files, generated binaries or disk images.

How it is marked: 10 points

Evidence for the lab objectives	Points
Correct wildcard explanation, and working hard and symbolic links	3
Rename failure, repair and recreated-name evidence	2
Inode identity and relative target resolution; original prediction and evidence-based correction	2
Individual checkpoint: result or diagnosis, and mechanism	2
Concise, attributable evidence and the required artifacts	1

Other valid commands and names earn credit. A naming difference costs points only when it breaks the result. If you use AI, note one useful suggestion and how you verified it.

Common mistakes

Mistake	What you see	Fix
Running ln again on an existing name	File exists	Look at the destination first
Target written from where you stand	A dangling link	Write it as if you stood next to the link
Replacing hard.txt to fix the link	The experiment loses its point	Repair only soft.txt
A bare pattern with rm	More files removed than expected	Run ls with the same pattern first
Brackets used like braces	A name that does not exist	[ ] match files, { } write words

GRUB and shared libraries

Optional extensions. They need a disposable virtual machine or a compiler, and they are not part of the two-hour core.

From power-on to login

Stage	What it does
BIOS or UEFI	Self-test, then finds the boot device
GRUB	Shows a menu and loads the kernel
Kernel	Starts the hardware, mounts the root filesystem, runs the first process (PID 1)
init or systemd	Starts the system services in order
Login	User space is ready

GRUB files and key settings

File	Purpose	Edit?
`/boot/grub/grub.cfg`	The generated menu	Never
`/etc/default/grub`	Timeout, kernel parameters	Yes
`/etc/grub.d/`	Scripts, custom entries	Yes
`/boot/vmlinuz-*`	The compressed kernel	No

Setting	Meaning	Example
`GRUB_DEFAULT`	Entry to boot	0
`GRUB_TIMEOUT`	Seconds at the menu	10
`GRUB_CMDLINE_LINUX_DEFAULT`	Kernel parameters	quiet splash
`GRUB_BACKGROUND`	A menu background image	/boot/grub/bg.png

After you edit /etc/default/grub, run sudo update-grub. Custom entries go in /etc/grub.d/40_custom. Do this only in a disposable virtual machine, never on the shared server.

GRUB recovery

Mode	How you enter	Use
Recovery mode	GRUB menu, Advanced options, recovery	A root shell, fsck, limited services
GRUB command line	Press c at the menu	Boot by hand when the menu is broken
Live USB	Boot from an external image	Full rescue when GRUB is gone

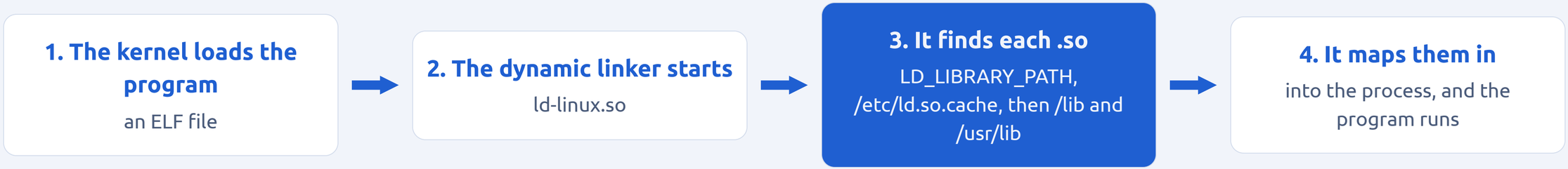


Take a snapshot of the virtual machine before you break anything on purpose.

Static and dynamic linking

	Static	Dynamic
When linked	At compile time	When the program starts
Library code	Copied into the executable	Loaded from .so files
Executable size	Larger	Smaller
Memory	Each program has its own copy	Programs share one copy
Updates	Recompile every program	Replace the .so, and all programs benefit
File extension	.a (archive)	.so (shared object)

How a program finds its libraries



Looking at shared libraries, and building one

student@server: ~/oslab-work/lab3

```
$ ldd /bin/ls
$ ldconfig -p | grep libc
$ gcc -fPIC -c mylib.c -o mylib.o
$ gcc -shared -o libmylib.so mylib.o
$ gcc main.c -L. -lmylib -o myprogram
$ LD_LIBRARY_PATH=. ./myprogram
```

ldd lists the libraries a program needs. ldconfig -p searches the library cache.

-fPIC makes position-independent code, so the library can load at any address. -shared makes the .so file.

For a user-local library, set LD_LIBRARY_PATH instead of installing into system folders.

More ways to look at a library

student@server: ~/oslab-work/lab3

```
$ file /lib/x86_64-linux-gnu/libc.so.6
$ readelf -h /bin/ls
$ grep '[.]so' /proc/$$/maps
$ sudo ldconfig
```

file tells you the kind of file: an ELF shared object.

readelf -h prints the ELF header of a program. /proc/$$/maps lists what the current shell has loaded.

After you install a new .so in a system folder, sudo ldconfig refreshes the cache. Not on the shared server.

Key takeaways

01

Wildcards

The shell expands a pattern into names before the command runs. Quote a pattern to keep it literal.

02

Name and inode

A file name is an entry that points to an inode. Hard links are more names for one inode.

03

Symbolic links

A symbolic link stores a path. A relative target is read from the link's folder.

04

Evidence

Rename, look, repair, replace. Each step has a record you can check.

Next session



COMING UP

Lab 4: I/O redirection, pipelines and processes

Send output to files, connect commands with pipes, and see and stop processes.



SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapters 13 and 14, File-System Interface and File-System Implementation.